Now that we have indexed a few more documents, we can get back on track and experiment with some other search features. Let's begin by searching for a word across multiple fields.

2.3.4 Searching across multiple fields

When a customer searches for something in a search bar, the search isn't necessarily restricted to just one field. For example, suppose we want to search for all documents where the word *Java* appears: not just in the `title` field, but also in other fields like `synopsis`, `tags`, and so on. To do this, we can enable a *multi-field* search.

Let's see an example of a query to search for *Java* in two fields: `title` and `synopsis`. Similar to the `match` query we saw earlier, Elasticsearch provides the `multi_match` query that will serve our purpose. We need to provide the search words in the inner `query` object, along with the fields we are interested in.

Listing 2.10 Searching across multiple fields

```
GET books/_search
{
  "query": {
    "multi_match": {
      "query": "Java",
      "fields": ["title","synopsis"]
    }
  }
}
```

Multi_match query that searches across multiple fields

Search words

Searches across two fields

When we execute this multi-field query, our results with the *Java* search word appear in both the `title` and `synopsis` fields. But say we want to bump up the priority of a result based on a particular field. For example, if *Java* is found in the `title` field, we want to boost that search result so it is three times as important and relevant while keeping the other documents at a normal priority. To do so, we can use the (naturally) boosting feature (users may be happy to see documents at the top of the list that have the search word in the title).

Relevancy scores

Full-text query results have numerical scoring values for documents, expressed as a `_score` attribute attached to the individual results. The `_score` is a positive floating-point number indicating how relevant the resultant document is to the query. The first document returned will have the highest score, and the last document will have the lowest. This is the *relevancy score*, and it indicates how well the documents matched the query. The higher the score, the greater the match.

Elasticsearch has an algorithm called Okapi Best Match 25 (BM25), which is an enhanced term frequency/inverse document frequency (TF/IDF) similarity algorithm that calculates the relevancy scores for the results and sorts them in that order when presenting them to the client.